

YOLO × FluxQ

AI-Powered Shipment Delivery Risk Score & Intelligent Recovery Platform

****Theme:**** Theme 4 — Logistics and Supply Chain

****Selected Problem Statement:**** PS 1 — Shipment Delivery Risk Score

****Project Category:**** Artificial Intelligence, Machine Learning, Predictive Analytics, Hybrid Optimization, Quantum Computing

****Project Type:**** Predictive and prescriptive logistics decision-support platform

1. Executive Summary

Modern logistics networks operate under uncertain conditions. Weather disruptions, traffic congestion, port delays, carrier unreliability, geopolitical incidents, and unexpected changes in transit conditions can cause shipments to miss committed delivery dates. Such disruptions affect customer satisfaction, service-level agreements (SLAs), transportation expenditure, inventory availability, and operational efficiency.

Traditional shipment visibility systems primarily report shipment location and status. Although useful, visibility alone does not always provide sufficient predictive intelligence to identify a potential delay before it occurs or determine the most suitable corrective action.

YOLO × FluxQ is proposed as an AI-powered Shipment Delivery Risk Intelligence and Recovery Platform. It continuously analyzes shipment information, historical transportation performance, and external disruption signals to predict delivery delays, generate explainable risk scores from 1 to 10, identify potential SLA breaches, and recommend feasible mitigation actions.

The platform combines machine learning for risk prediction, ETA forecasting for delay estimation, explainable AI for identifying the factors behind a risk, and constrained optimization for selecting recovery plans. A hybrid quantum-classical optimization module is proposed for small combinatorial allocation problems where multiple disrupted shipments compete for limited alternative transportation capacity.

Unlike a passive alerting system, YOLO × FluxQ follows a closed-loop decision-support approach:

****Sense → Predict → Explain → Optimize → Recommend → Replan****

When new information arrives, the system updates the shipment risk, reevaluates the delivery impact, and recalculates recovery recommendations. Operators retain approval authority for consequential actions.

The proposed platform will be demonstrated through a simulated logistics network containing shipment records, route conditions, external disruption events, carrier alternatives, delivery commitments, and limited recovery resources.

The intended outcome is a practical decision-support system that helps logistics teams identify high-risk shipments earlier, prioritize interventions, evaluate operational trade-offs, and respond to disruptions with greater speed and consistency.

2. Problem Statement

2.1 Official problem context

Shipment delivery operations are affected by disruptions such as:

- Weather events.
- Traffic congestion.
- Port congestion.
- Geopolitical incidents.
- Other external transportation factors.

Existing systems may provide shipment visibility but lack predictive intelligence to identify risks before the impact reaches delivery operations.

This can result in:

- SLA breaches.
- Increased operational costs.
- Reactive decision-making.
- Inefficient resource allocation.
- Reduced customer experience.

2.2 Selected problem statement

Shipment Delivery Risk Score

The proposed platform must continuously ingest real-time and predictive signals, including weather, traffic, port congestion, shipment progress, and historical performance, to:

1. Generate a shipment risk score between 1 and 10.
2. Predict potential SLA breaches.
3. Identify shipment disruption risks before they materially affect delivery.
4. Provide actionable mitigation recommendations.
5. Improve delivery reliability and operational decision-making.

2.3 Problem interpretation

The project treats the problem as two connected tasks.

Predictive task

Determine:

- Whether a shipment is likely to be delayed.
- How much delay may occur.
- Whether the committed delivery time may be missed.

- Which factors contribute to the risk.
- How the risk changes as new information arrives.

Prescriptive task

Determine:

- Whether intervention is necessary.
- Which feasible recovery options are available.
- Which option offers the most appropriate trade-off between delivery time, cost, SLA protection, and resource usage.
- Whether the current plan should be retained, modified, or escalated for human review.

The first task answers ****what is likely to happen****.

The second answers ****what can be done about it****.

3. Project Objectives

3.1 Primary objectives

1. Build a predictive shipment-risk engine using historical and current shipment features.
2. Generate a standardized risk score from 1 to 10.
3. Estimate updated shipment ETA and delay duration.
4. Predict potential SLA breaches.
5. Explain the major factors contributing to each shipment's risk.
6. Recommend feasible mitigation actions.
7. Recalculate recommendations when new disruption events occur.
8. Provide an interactive logistics control tower for monitoring and decision support.

3.2 Advanced objectives

1. Model limited carrier and route capacity as an optimization problem.
2. Generate and compare multiple feasible recovery plans.
3. Use classical optimization as a reliable baseline.
4. Experiment with a quantum-hybrid solver on a suitably small combinatorial subproblem.
5. Measure solution quality and runtime against the classical approach.
6. Maintain human approval and decision traceability.

3.3 Success criteria

The project will be evaluated using predictive accuracy, ETA error, risk calibration, feasibility of recommended plans, response time, and simulated operational outcomes.

No specific improvement percentage will be claimed before the prototype is tested.

4. Proposed Solution

YOLO × FluxQ is a shipment decision-intelligence platform that transforms transportation signals into predictive risk assessments and actionable recovery recommendations.

The platform consists of six functional layers.

4.1 Data ingestion layer

Collects and normalizes:

- Shipment details.
- Origin and destination.

- Planned route.
- Current shipment progress.
- Promised delivery time.
- Historical transit performance.
- Weather and traffic signals.
- Port or terminal congestion.
- Carrier reliability.
- Available alternate transportation capacity.

For the hackathon prototype, external signals may be simulated or obtained through permitted public data sources. Mock carrier and logistics APIs will represent enterprise integrations.

4.2 Predictive intelligence layer

Uses machine learning to estimate:

- Probability of delay.
- Probability of SLA breach.
- Predicted ETA.
- Expected delay duration.
- Risk score from 1 to 10.

4.3 Explainability layer

Identifies the main factors contributing to the risk assessment.

Example:

****Shipment SH-2048 — Risk 8.4/10****

Contributing factors:

- Severe congestion on the current route.
- Historical delay frequency on the route.
- Reduced remaining SLA buffer.
- Carrier performance below the lane average.

These are illustrative values. Actual explanations will be generated from the trained model.

4.4 Disruption impact layer

Evaluates how an event affects the shipment and its delivery commitment.

Examples:

- A traffic incident increases estimated transit time.
- A port congestion update changes the expected departure time.
- A carrier capacity reduction removes an alternate option.
- A new ETA prediction changes the SLA breach probability.

4.5 Recovery optimization layer

Evaluates available recovery actions under operational constraints.

Possible actions include:

- Retain the original route.
- Reroute through an alternate path.
- Assign an alternate carrier.
- Expedite transportation.
- Reallocate limited recovery capacity.
- Escalate the shipment to an operator.

4.6 Decision and monitoring layer

Presents:

- Shipment risk board.
- Risk score and probability.
- Predicted ETA.
- Potential SLA impact.
- Risk explanations.
- Recommended action.
- Alternative plans.
- Estimated cost and delay trade-offs.
- Operator approval.
- Decision history.

5. YOLO × FluxQ Conceptual Foundation

The project uses YOLO × FluxQ as the combined project identity.

The FluxQ concepts retained in this proposal are:

- Sense → Predict → Decide → Execute → Adapt.
- Early Warning Indicators.
- Dynamic replanning.
- Multiple feasible alternative plans.
- Disruption-to-constraint translation.
- Classical and quantum-hybrid optimization.
- Quantification of the quantum contribution.

- Human-in-the-loop decision approval.

The exact technical meaning of any separate YOLO model or framework should be confirmed against its original source before attributing additional capabilities to it. This proposal does not assume that the name YOLO means a computer-vision detector.

5.1 Core operating principle

A shipment is not treated as a static record. It is treated as a changing operational state.

For each shipment, the system maintains:

- Current estimated state.
- Current risk assessment.
- Predicted delivery outcome.
- Available recovery actions.
- Selected recommendation.
- Latest event that affected the decision.

A new event triggers reassessment rather than leaving the earlier prediction unchanged.

6. Functional Requirements

FR-01: Shipment registration

The system shall allow shipment records to be created or imported.

Each shipment shall include a unique ID, origin, destination, carrier, planned route, dispatch time, promised delivery time, and current status.

FR-02: Signal ingestion

The system shall ingest shipment updates and external signals from mock APIs, uploaded datasets, or permitted data feeds.

FR-03: Risk prediction

The system shall calculate a delay probability and an SLA-breach probability using the predictive model.

FR-04: Risk score

The system shall convert the calibrated SLA-breach probability into an integer risk score from 1 to 10.

FR-05: ETA prediction

The system shall estimate the expected delivery time and delay duration.

FR-06: Risk explanation

The system shall show the primary factors contributing to a shipment's predicted risk.

FR-07: Disruption detection

The system shall detect a new event or meaningful change in route, weather, traffic, congestion, carrier status, or shipment progress.

FR-08: Mitigation generation

The system shall evaluate available recovery actions and produce feasible alternatives.

FR-09: Plan comparison

The system shall compare candidate plans using delivery time, incremental cost, SLA outcome, resource usage, and other configured objectives.

FR-10: Dynamic replanning

The system shall recalculate affected recommendations when new information changes the problem constraints.

FR-11: Human approval

The system shall allow an authorized operator to approve, reject, or defer a proposed action.

FR-12: Audit trail

The system shall record the prediction, event, recommendation, approval decision, and resulting outcome.

7. Non-Functional Requirements

Reliability

The system should preserve a valid classical recommendation when the advanced optimizer is unavailable or fails to produce a feasible result.

Explainability

Risk scores and recovery recommendations should be accompanied by understandable reasons.

Responsiveness

The prototype should process simulated events and refresh its dashboard within an agreed demo response-time target.

Security

The application should use role-based access for operator functions and avoid exposing credentials in source code.

Scalability

The architecture should separate ingestion, prediction, optimization, and presentation so individual components can be upgraded independently.

Data integrity

Shipment IDs, timestamps, route states, and event records should be validated before model inference or optimization.

Human oversight

The platform shall remain a decision-support system. It will not autonomously execute real transportation changes in the hackathon prototype.

8. System Architecture

8.1 Logical architecture

Layer 1 — Data sources

- Shipment database.
- Historical shipment outcomes.
- Weather feed or simulator.
- Traffic feed or simulator.
- Port congestion feed or simulator.
- Carrier and route capacity records.

Layer 2 — Data processing

- Schema validation.
- Timestamp normalization.
- Missing-value handling.
- Feature construction.
- Event deduplication.
- Shipment-state updates.

Layer 3 — Predictive services

- Delay classification.
- SLA-breach classification.
- ETA regression.
- Risk-score calculation.
- Explainability.

Layer 4 — Optimization services

- Feasibility filtering.
- Candidate action generation.
- Classical optimization.
- Optional QUBO formulation.
- QAOA or simulator experiment.

- Candidate validation.
- Baseline comparison.

Layer 5 — Application services

- Shipment APIs.
- Risk APIs.
- Event APIs.
- Recommendation APIs.
- Approval APIs.
- Audit APIs.

Layer 6 — User interface

- Control tower.
- Shipment details.
- Risk explanation.
- Recovery plan comparison.
- Event simulation.
- Decision history.

8.2 Suggested technology stack

Component	Proposed technology
---	---
Frontend	React + TypeScript
UI styling	Tailwind CSS
Backend	Python + FastAPI
Data processing	Pandas, NumPy
ML	scikit-learn, XGBoost or LightGBM
Explainability	SHAP

Classical optimization	Google OR-Tools
Quantum experiment	Qiskit QAOA or a simulator
Database	PostgreSQL; SQLite for a lightweight demo
Charts	Recharts
Map	Leaflet with OpenStreetMap
Event simulation	Python event generator
API communication	REST/JSON
Deployment	Docker and a suitable application host

The stack is a proposed implementation choice, not a requirement stated by the problem statement.

9. Data Design

9.1 Shipment entity

Field	Type	Description
shipment_id	String	Unique shipment identifier
origin	String	Origin location
destination	String	Destination location
carrier_id	String	Assigned carrier
route_id	String	Current planned route
dispatch_time	Datetime	Actual dispatch time
promised_eta	Datetime	Committed delivery time
current_eta	Datetime	Latest predicted arrival
status	Enum	Created, in transit, delayed, delivered, etc.
priority	Integer	Operational priority
shipment_type	String	Shipment category

| weight | Float | Shipment weight |
| volume | Float | Shipment volume |

9.2 External signal entity

Field	Type	Description
event_id	String	Unique event ID
event_type	Enum	Weather, traffic, port, carrier, other
location_id	String	Affected location or segment
severity	Numeric	Event severity
start_time	Datetime	Event start
expected_end	Datetime	Expected end
confidence	Float	Source confidence
source	String	Feed or simulator

9.3 Carrier entity

Field	Type	Description
carrier_id	String	Carrier identifier
lane_id	String	Supported route or lane
historical_on_time_rate	Float	Historical performance
capacity_available	Float	Available capacity
cost_factor	Float	Relative transportation cost
service_type	String	Standard, expedited, etc.

9.4 Shipment outcome entity

Historical records should include:

- Actual delivery timestamp.
- Actual transit duration.
- Whether delivery was late.
- Delay duration.
- Whether SLA was breached.
- Carrier and route used.
- Recorded disruption conditions.

The historical target labels must be derived from actual outcomes, not from the model's own predictions.

9.5 Dataset strategy

For a hackathon prototype, use:

1. Public transportation or weather data where suitable and permitted.
2. Synthetic shipment records.
3. Simulated disruption events.
4. Clearly labeled illustrative carrier capacity and cost values.

Do not present synthetic records as actual company or Hyundai Mobis data.

10. AI/ML Methodology

10.1 Prediction tasks

The predictive system has three distinct outputs.

Task A: Delay probability

Estimate:

$$\begin{aligned} & \backslash[\\ & P(\text{\texttt{Late}} \mid X) \\ & \backslash] \end{aligned}$$

Where $\backslash(X\backslash)$ represents the shipment and current operating conditions.

Task B: SLA-breach probability

Estimate:

$$\begin{aligned} & \backslash[\\ & P(\text{\texttt{SLA Breach}} \mid X) \\ & \backslash] \end{aligned}$$

This is the probability used to derive the official shipment risk score.

Task C: ETA regression

Estimate the expected arrival time:

$$\begin{aligned} & \backslash[\\ & \widehat{\text{ETA}} = f(X) \\ & \backslash] \end{aligned}$$

Expected delay:

$$\begin{aligned} & \backslash[\\ & \widehat{D} = \max(0, \widehat{\text{ETA}} - \text{ETA}_{\text{\texttt{promised}}}) \end{aligned}$$

\]

All time values must use a consistent timezone and timestamp convention.

10.2 Feature engineering

Shipment features

- Origin-destination lane.
- Shipment type.
- Weight and volume.
- Dispatch hour.
- Day of week.
- Remaining distance.
- Remaining planned transit time.
- Current transit progress.

Route features

- Historical average transit time.
- Historical delay frequency.
- Current traffic level.
- Number of affected route segments.
- Alternate route availability.

External features

- Weather severity.
- Rainfall or adverse weather indicator.
- Traffic congestion.
- Port congestion.

- Disruption duration.
- Signal confidence.

Carrier features

- Historical on-time rate.
- Historical lane-specific performance.
- Available capacity.
- Service class.

SLA features

- Remaining SLA buffer.
- Predicted ETA deviation.
- Time remaining until commitment.

Features that would only become available after delivery must not be used to predict earlier risk. This avoids data leakage.

10.3 Model selection

Classification model

Recommended starting point: XGBoost or LightGBM.

Reason:

- Handles nonlinear relationships.
- Works well with tabular features.
- Supports feature importance and explainability workflows.
- Can be evaluated against simpler baselines.

Baseline models:

- Logistic regression.
- Decision tree.
- Random forest.

The final model should be selected using validation results, not assumed superiority.

ETA model

Possible models:

- Gradient boosting regression.
- Random forest regression.
- A simple historical lane-duration baseline.

A sophisticated model should only be retained if it improves measured ETA error.

10.4 Training process

1. Validate source data.
2. Remove duplicate records.
3. Normalize timestamps and units.
4. Create historical route and carrier features.
5. Derive target labels from actual outcomes.
6. Split training and test data by time where feasible.
7. Train baseline models.
8. Train candidate models.
9. Calibrate probabilities.
10. Evaluate performance.

11. Save the selected model and preprocessing pipeline.
12. Monitor prediction quality as new outcomes become available.

10.5 Risk-score calculation

Let:

$$p = P(\text{SLA breach})$$

The base risk score is:

$$R = \max(1, \min(10, \lceil 10p \rceil))$$

This creates a 1–10 score while preserving the underlying probability as a separate output.

Example mapping:

Probability	Score
0%–10%	1
>10%–20%	2
>20%–30%	3
>30%–40%	4
>40%–50%	5
>50%–60%	6
>60%–70%	7
>70%–80%	8

| >80%–90% | 9 |

| >90%–100% | 10 |

A score of 1 does not mean zero risk. It means the lowest risk band.

The thresholds may be adjusted after calibration and operational review, but the published score must remain interpretable.

10.6 Risk categories

For dashboard readability:

- 1–3: Low.
- 4–6: Moderate.
- 7–8: High.
- 9–10: Critical.

These categories are proposed presentation bands, not official categories specified in the PS.

10.7 Explainable AI

Use SHAP or a comparable explanation method to identify influential features.

An explanation should distinguish:

- Factors increasing predicted risk.
- Factors reducing predicted risk.
- Data uncertainty.
- Whether the feature is directly observed or estimated.

Example:

“Risk is elevated primarily because the predicted ETA exceeds the committed time, the current route has elevated congestion, and the remaining SLA buffer is small.”

The application must not describe correlation as proof of causation.

11. Early Warning Indicator Engine

The Early Warning Indicator (EWI) identifies a meaningful change that may require intervention.

An EWI may be triggered when:

1. SLA-breach probability crosses a configured threshold.
2. Predicted ETA moves beyond the promised delivery time.
3. Risk score increases materially between updates.
4. A disruption affects the current route.
5. A carrier becomes unavailable.
6. An alternate route becomes infeasible.
7. A high-priority shipment enters a critical risk band.

11.1 Example event

Before event:

- SLA-breach probability: 22%.
- Risk score: 3.
- ETA: 15:30.
- Promised delivery: 17:00.

After congestion update:

- SLA-breach probability: 78%.
- Risk score: 8.
- ETA: 19:10.
- Promised delivery: 17:00.

The EWI marks the shipment for recovery evaluation.

The system should store the previous and current predictions so that operators can see how the risk changed.

12. Disruption Impact Engine

The disruption engine converts an incoming event into an updated operational problem.

12.1 Event processing

1. Receive event.
2. Validate event schema.
3. Check timestamp and source.
4. Identify affected route segments or locations.
5. Identify shipments currently using affected segments.
6. Update relevant shipment features.
7. Recalculate ETA and risk.
8. Determine whether recovery evaluation is required.

12.2 Disruption compiler

The disruption compiler translates an event into changed optimization inputs.

Example:

****Event:**** Road segment unavailable for four hours.

Changes:

- Original route marked infeasible or penalized.
- Expected transit time updated.
- Alternate routes evaluated.
- Carrier capacity checked.
- Affected shipments prioritized.
- Recovery plan recalculated.

The event itself does not dictate the recovery action. It changes the conditions under which the optimizer evaluates alternatives.

13. Intelligent Recovery Optimization

This module extends the official risk-scoring requirement into actionable mitigation.

13.1 Decision variables

Let:

$\mathcal{V}$

$x_{\{i,a\}} =$

$\begin{cases}$

$$\begin{aligned}
 &1 \text{ \& \text{if shipment } i \text{ receives action } a} \\
 &0 \text{ \& \text{otherwise}} \\
 &\text{\end{cases}} \\
 &\text{\]}
 \end{aligned}$$

Each shipment should receive at most one selected recovery action:

$$\begin{aligned}
 &\text{\[} \\
 &\sum_{a \in A_i} x_{i,a} = 1 \\
 &\text{\]}
 \end{aligned}$$

Where A_i is the set of available actions for shipment i .

13.2 Objective function

A simplified multi-objective cost is:

$$\begin{aligned}
 &\text{\[} \\
 &\min \sum_i \sum_a x_{i,a} \\
 &\text{\left(} \\
 &\alpha C_{i,a} \\
 &+\beta D_{i,a} \\
 &+\gamma B_{i,a} \\
 &+\delta E_{i,a} \\
 &\text{\right)} \\
 &\text{\]}
 \end{aligned}$$

Where:

- $C_{i,a}$: incremental transportation cost.
- $D_{i,a}$: expected delivery delay.

- $\backslash(B_{i,a})$: SLA-breach penalty.
- $\backslash(E_{i,a})$: emissions or other operational impact.
- $\backslash(\alpha, \beta, \gamma, \delta)$: configurable weights.

The weights should be selected and documented for the demo scenario. They are not universal values.

13.3 Constraints

Capacity

$$\backslash[\sum_i q_i x_{i,a} \leq Cap_a \backslash]$$

Where $\backslash(q_i)$ is shipment demand on resource $\backslash(a)$, and $\backslash(Cap_a)$ is available capacity.

Feasibility

An action that violates a route, carrier, or capacity constraint must not be selected.

Deadline

The system may penalize or prioritize plans that fail to meet committed delivery time.

Budget

If a recovery budget is configured:

$$\backslash[\sum_i \sum_a C_{i,a} x_{i,a} \leq Budget \backslash]$$

\]

Priority

High-priority shipments may receive a higher SLA penalty or a defined service priority.

13.4 Candidate recovery actions

Action	Description	Trade-off
--------	-------------	-----------

---	---	---
-----	-----	-----

Maintain route	Continue existing plan	Lowest intervention cost, potentially higher delay
----------------	------------------------	--

Alternate route	Use another feasible path	May reduce delay, may increase distance
-----------------	---------------------------	---

Carrier switch	Assign another carrier	Depends on availability and cost
----------------	------------------------	----------------------------------

Expedite	Use faster service	Higher cost
----------	--------------------	-------------

Hold and monitor	Defer action until more evidence	Avoids premature intervention, risks lost response time
------------------	----------------------------------	---

Escalate	Send to operator	Human judgment required
----------	------------------	-------------------------

13.5 Multiple feasible plans

The system may generate several feasible candidate plans and rank them according to the configured objective.

For example:

- Plan A: Lowest incremental cost.
- Plan B: Lowest expected delay.
- Plan C: Lowest SLA-breach risk.
- Plan D: Balanced operational trade-off.

The platform should show the trade-offs rather than label one plan universally best.

14. Quantum-Hybrid Optimization Module

14.1 Purpose

Quantum computing is not used for the risk classification model or ETA prediction.

Its proposed role is to experiment with a small combinatorial optimization problem, such as assigning several disrupted shipments to limited alternative carrier slots.

14.2 Problem formulation

For a small set of shipments and recovery resources, binary variables represent assignment choices.

A QUBO objective can be expressed as:

$$\begin{aligned} & \backslash[\\ & E(x) = E_{\text{cost}}(x) + \\ & \lambda E_{\text{constraint}}(x) \\ & \backslash] \end{aligned}$$

Where:

- E_{cost} captures recovery cost and delay penalties.
- $E_{\text{constraint}}$ penalizes invalid assignments.
- λ is a penalty coefficient.

The QUBO is mapped to an Ising Hamiltonian for QAOA or an appropriate quantum optimization backend.

14.3 Hybrid workflow

1. Classical preprocessing removes impossible assignments.
2. Classical optimization generates a feasible baseline.
3. A small QUBO is constructed from the reduced problem.
4. QAOA or a quantum simulator produces candidate bitstrings.
5. Candidate solutions are decoded.
6. A classical validator checks feasibility.
7. Valid candidates are scored using the original operational objective.
8. The system compares candidate quality and runtime with the classical baseline.
9. The best feasible result is retained.

14.4 Quantum Contribution Ratio

Define a transparent experimental measure:

$$\begin{aligned} & \backslash[\\ & \text{QCR=} \\ & \frac{J_{\text{classical}} - J_{\text{hybrid}}}{J_{\text{classical}}} \\ & \times 100 \\ & \backslash] \end{aligned}$$

Where J is the same objective value for both solutions.

Interpretation:

- Positive QCR: hybrid candidate has a lower objective.
- Zero QCR: equal objective.
- Negative QCR: hybrid candidate is worse.

This metric should only be reported when both solutions are feasible and evaluated using the same objective and constraints.

Also report:

- Runtime.
- Feasibility rate.
- Number of trials.
- Problem size.
- Solver settings.
- Whether the quantum backend is simulated or hardware-based.

Do not claim quantum advantage unless a suitable benchmark establishes it.

14.5 Fallback strategy

If the quantum solver is unavailable, slow, or infeasible, the platform uses the classical solution.

This ensures that the product remains functional without relying on quantum execution.

15. Dynamic Replanning Engine

Dynamic replanning is one of the central FluxQ concepts.

15.1 Why replanning is needed

A recovery plan is based on current conditions. If the conditions change, the original recommendation may no longer be valid.

Examples:

- Alternate route becomes congested.
- Carrier withdraws capacity.
- Weather improves.
- Shipment progress is faster than expected.
- A new high-priority shipment enters the network.

15.2 Replanning procedure

1. Receive the new event.
2. Determine which shipments and constraints are affected.
3. Recompute ETA and risk.
4. Preserve unaffected decisions where appropriate.
5. Invalidate infeasible candidate plans.
6. Recalculate the affected optimization subproblem.
7. Compare new alternatives with the previous recommendation.
8. Notify the operator if a material decision change occurs.

The system should avoid unnecessary complete replanning when only a small part of the network has changed, while preserving feasibility and correctness.

16. Application Modules

Module 1: Logistics Control Tower

Displays:

- Total active shipments.

- Risk distribution.
- Critical shipments.
- Predicted SLA breaches.
- Recent disruption events.
- Pending recovery recommendations.
- Shipment status and trend.

Module 2: Shipment Risk Board

Each shipment card or table row contains:

- Shipment ID.
- Origin and destination.
- Carrier.
- Current ETA.
- Promised ETA.
- Risk score.
- Breach probability.
- Risk category.
- Latest event.

Filters include risk category, carrier, route, status, and priority.

Module 3: Shipment Intelligence Detail

Shows:

- Shipment timeline.
- Current route.
- ETA history.
- Risk-score history.

- Prediction confidence or calibration information.
- Main risk drivers.
- Relevant disruption events.

Module 4: Recovery Recommendation Center

Displays candidate actions with:

- Expected arrival.
- Predicted delay.
- SLA outcome.
- Incremental cost.
- Capacity usage.
- Feasibility status.
- Recommendation rationale.

Module 5: Disruption Simulator

Allows a demo operator to trigger:

- Traffic congestion.
- Severe weather.
- Port delay.
- Carrier capacity reduction.
- Route closure.

The simulator updates the shipment state and initiates reassessment.

Module 6: Hybrid Optimization Comparison

Shows:

- Classical baseline.
- Hybrid candidate.
- Objective values.
- Runtime.
- Feasibility.
- QCR.
- Solver status.

Module 7: Decision Audit

Stores:

- Original prediction.
- Triggering event.
- Updated prediction.
- Recommended plan.
- Operator decision.
- Final outcome.

17. API Design

Proposed REST endpoints:

Method	Endpoint	Purpose
---	---	---
GET	/shipments`	List shipments
POST	/shipments`	Create shipment
GET	/shipments/{id}`	Shipment detail

GET	`/shipments/{id}/risk`	Risk and probability
GET	`/shipments/{id}/eta`	ETA estimate
GET	`/shipments/{id}/explanation`	Risk explanation
POST	`/events`	Submit disruption event
POST	`/risk/recompute`	Recalculate affected risks
POST	`/recovery/optimize`	Generate recovery plans
GET	`/recovery/{id}`	Retrieve candidate plans
POST	`/recovery/{id}/approve`	Approve recommendation
POST	`/recovery/{id}/reject`	Reject recommendation
GET	`/optimization/benchmark`	Retrieve solver comparison
GET	`/audit/{shipment\_id}`	Decision history

All approval endpoints should validate the user's authorization and the current status of the recommendation.

18. End-to-End Operational Example

The following example uses illustrative data for the prototype.

Initial shipment state

Shipment: SH-2048

- Origin: Chennai.
- Destination: Bengaluru.
- Carrier: Carrier A.
- Promised ETA: 18:00.
- Current predicted ETA: 17:20.
- SLA-breach probability: 18%.

- Risk score: 2.

The system initially recommends monitoring without intervention.

New disruption

At 13:00, a simulated severe congestion event affects the planned route.

The event is ingested and matched to the shipment's current route.

Updated prediction

- Predicted ETA: 20:15.

- SLA-breach probability: 82%.

- Risk score: 9.

- Predicted delay: 2 hours 15 minutes.

The EWI marks the shipment as critical.

Recovery alternatives

Plan	Predicted ETA	Additional cost	SLA result
------	---------------	-----------------	------------

---	---	---	---
-----	-----	-----	-----

Maintain original route	20:15	₹0	Breach likely
-------------------------	-------	----	---------------

Alternate route B	17:50	₹1,200	Within commitment
-------------------	-------	--------	-------------------

Expedite with Carrier B	17:30	₹2,500	Within commitment
-------------------------	-------	--------	-------------------

The optimizer evaluates cost, delivery time, and resource feasibility.

The operator sees the alternatives and approves one.

Second event

At 14:00, the alternate route becomes congested.

The system:

1. Receives the new event.
2. Updates the affected route.
3. Recalculates ETA and risk.
4. Checks whether the approved plan remains feasible.
5. Generates a revised recommendation.
6. Requests operator approval if a material change is needed.

This scenario demonstrates the complete predictive and adaptive behavior.

19. User Roles

Logistics Operator

- Monitor shipment risks.
- View explanations.
- Review recovery alternatives.
- Approve or reject recommendations.
- Inspect shipment history.

Operations Manager

- View network-level risk.
- Configure risk thresholds and objective weights.

- Review service performance.
- Inspect optimization results.

System Administrator

- Manage data sources.
- Configure mock integrations.
- Monitor model and API health.
- Manage access.

The hackathon prototype may implement only operator and administrator roles if time is limited.

20. Evaluation Framework

The project will be evaluated across four dimensions.

20.1 Prediction quality

Classification

- Precision.
- Recall.
- F1 score.
- PR-AUC.
- Confusion matrix.

Recall is important because missed high-risk shipments can lead to unanticipated SLA breaches. Precision matters because excessive false alerts can overload operations teams.

Calibration

- Brier score.
- Reliability curve.

A risk probability should be meaningful, not merely a ranking score.

ETA accuracy

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\text{ETA}_i - \widehat{\text{ETA}}_i|$$

Report ETA error in minutes or hours.

20.2 Operational value

Compare:

1. No intervention.
2. Fixed-rule mitigation.
3. ML risk + classical optimization.
4. ML risk + hybrid optimization experiment.

Measure:

- Simulated SLA breaches.
- Total recovery cost.
- Mean delivery delay.
- Number of feasible recommendations.
- Time to generate a recommendation.

20.3 Optimization quality

Measure:

- Objective value.
- Constraint violations.
- Feasibility rate.
- Runtime.
- Difference from classical baseline.

20.4 User experience

Evaluate whether an operator can:

- Identify the most urgent shipments.
- Understand why a shipment is risky.
- Compare recovery options.
- Approve or reject a recommendation.
- Understand what changed after a new event.

21. Testing Strategy

Unit testing

Test:

- Risk-score boundaries.
- Probability-to-score mapping.

- ETA calculations.
- Timestamp handling.
- Constraint validation.
- Event deduplication.
- API validation.

Model testing

Test:

- Missing values.
- Unseen carriers.
- Unseen route categories.
- Class imbalance.
- Data leakage.
- Probability calibration.
- Temporal generalization.

Optimization testing

Test:

- No feasible alternate route.
- Insufficient capacity.
- Multiple shipments competing for one slot.
- High-priority shipment.
- Cost budget exceeded.
- Invalid quantum candidate.
- Classical fallback.

Integration testing

Test:

- Event ingestion to risk refresh.
- Risk refresh to recovery generation.
- Operator approval to audit record.
- New event to dynamic replanning.

Scenario testing

Test at least these cases:

1. No disruption.
2. Moderate traffic.
3. Severe congestion.
4. Weather disruption.
5. Carrier capacity reduction.
6. Multiple simultaneous shipment disruptions.
7. No feasible recovery option.
8. Quantum solver unavailable.
9. Event arrives after a recommendation has been approved.

22. Security, Privacy, and Responsible AI

The platform should:

- Use synthetic or authorized data.
- Avoid embedding API keys in frontend code.
- Validate and sanitize incoming event data.

- Restrict approval actions to authorized users.
- Log changes to operational decisions.
- Show uncertainty when model confidence is weak.
- Avoid presenting a prediction as a guaranteed outcome.
- Allow operators to override recommendations.
- Avoid using sensitive customer attributes without a justified and authorized purpose.
- Monitor prediction quality across routes and carriers.

The platform should not automatically penalize a carrier or customer based solely on an unverified model prediction.

23. Implementation Plan

Phase 1 — Foundation

- Define shipment schema.
- Create synthetic shipment dataset.
- Build FastAPI backend.
- Create database.
- Build initial dashboard.

Phase 2 — Predictive engine

- Engineer shipment features.
- Train baseline classifier.
- Train candidate ML model.
- Train ETA regressor.
- Calibrate breach probabilities.
- Implement 1–10 score.

- Add explanation output.

Phase 3 — Disruption simulation

- Create event generator.
- Match events to affected routes.
- Recalculate shipment predictions.
- Implement EWI triggers.

Phase 4 — Recovery optimization

- Define recovery actions.
- Implement feasibility constraints.
- Build OR-Tools baseline.
- Generate alternative plans.
- Display cost, ETA, and SLA trade-offs.

Phase 5 — Quantum-hybrid experiment

- Reduce a small assignment problem.
- Formulate QUBO.
- Run QAOA or simulator.
- Validate solutions.
- Compare against classical baseline.
- Display QCR and runtime.

Phase 6 — Dynamic replanning

- Detect material changes.
- Recompute affected recommendations.
- Preserve valid decisions.

- Add approval and audit workflow.

Phase 7 — Final demo and validation

- Run end-to-end scenarios.
- Validate model metrics.
- Test fallback behavior.
- Prepare architecture and benchmark documentation.
- Demonstrate the system with clearly labeled simulated data.

24. Project Deliverables

The proposed completed prototype should contain:

1. Working shipment control tower.
2. Shipment risk board with 1–10 scores.
3. ML prediction service.
4. ETA prediction service.
5. Explainable risk details.
6. Disruption event simulator.
7. SLA-breach detection.
8. Recovery recommendation engine.
9. Classical optimization baseline.
10. Quantum-hybrid optimization experiment.
11. Dynamic replanning.
12. Operator approval workflow.
13. Decision audit history.
14. Evaluation report.
15. Technical documentation.

16. Demo scenario and test dataset.

25. Limitations

Data availability

Predictive performance depends on representative historical shipment outcomes. Synthetic data can demonstrate functionality but cannot establish real-world predictive accuracy.

External signal quality

Weather, traffic, and congestion sources may be delayed, incomplete, or uncertain. Signal confidence should be preserved.

ETA uncertainty

ETA predictions are estimates. Unexpected events may still cause delivery outcomes to differ.

Optimization assumptions

Cost, capacity, travel-time, and SLA penalties must reflect the configured scenario. Incorrect assumptions can produce misleading recommendations.

Quantum limitations

QAOA and quantum simulators may not outperform classical solvers, especially on small problems. The quantum component is an experimental comparison, not a prerequisite for core functionality.

Integration limitations

The hackathon prototype uses mock or permitted data interfaces. It does not claim integration with proprietary enterprise systems.

26. Future Scope

Potential extensions include:

- Live carrier and transportation management integrations.
- More accurate route-level ETA prediction.
- Multimodal transportation support.
- Network-wide disruption propagation.
- Inventory impact estimation for delayed shipments.
- Customer notification recommendations.
- More advanced multi-objective optimization.
- Model drift detection and retraining.
- Larger-scale solver benchmarking.
- Enterprise role and approval policies.
- Integration with existing ERP, TMS, and supply-chain platforms.

These are future extensions, not necessary conditions for the initial PS 1 implementation.

27. Expected Impact

YOLO × FluxQ is designed to help logistics operations move from reactive shipment monitoring toward predictive and prescriptive decision support.

Potential benefits to be measured include:

- Earlier identification of high-risk shipments.
- Better visibility into potential SLA breaches.
- More consistent mitigation decisions.
- Improved use of limited alternate transportation capacity.
- Reduced avoidable delay and recovery expenditure.
- Faster response to changing disruptions.
- More transparent operational decision-making.

The project will validate these benefits through simulation and measured prototype results rather than claiming unverified improvements.

28. Conclusion

YOLO × FluxQ addresses the Shipment Delivery Risk Score problem by combining predictive intelligence with actionable recovery optimization.

The machine-learning layer estimates delivery risk and ETA. Explainable AI communicates why a shipment is at risk. The disruption engine translates new events into updated operational conditions. Classical optimization generates feasible recovery plans, while a quantum-hybrid experiment evaluates a small combinatorial allocation problem against a classical baseline. Dynamic replanning allows recommendations to evolve as the transportation environment changes.

The result is a focused logistics decision-support platform that goes beyond displaying shipment status: it helps operators understand risk, evaluate possible responses, and make informed recovery decisions.

****Final project identity:****

****YOLO × FluxQ — Predictive Shipment Risk Intelligence & Hybrid Recovery Optimization****

****Selected problem statement:** Theme 4, PS 1 — Shipment Delivery Risk Score.**

****Core value proposition:**** Predict shipment risk before delivery failure, explain the cause, recommend a feasible recovery action, and adapt when conditions change.